Product Requirements Document (PRD)

AI-Powered Security Investigation & Evidence Analysis Platform

“[illegible].”

Default language: Arabic (RTL) | **Secondary:** English (LTR) | **Backend:** Python | **Frontend:** HTML / CSS / JavaScript | **LLM Gateway:** OpenRouter API

1. Product Overview

Mirsad (■■■■■■) is a simple, user-friendly AI security investigation application for analyzing suspicious emails, links, files, screenshots/images, SIEM exports, and security alerts.

The product presents an AI investigator that behaves as a highly skilled and careful security analyst. It should not merely summarize input: it investigates evidence, identifies what is suspicious, explains why it matters, highlights gaps, and recommends what the user should investigate or do next.

The default experience is Arabic with full RTL support; English is available as a complete alternative language.

2. Problem Statement

SOC analysts and ordinary users can face large volumes of phishing emails, suspicious links, potentially malicious files, and noisy SIEM alerts. A user may know that something looks suspicious but still need an accurate assessment of whether it is genuinely dangerous, what evidence supports that conclusion, and where to investigate deeper.

Mirsad addresses this gap by turning raw evidence into an understandable investigation: what happened, what is dangerous, what evidence was found, how confident the system is, what remains unknown, and which next steps are most useful.

3. Target Users

Primary: SOC L1 analysts who need help triaging alerts, understanding evidence, and deciding where to investigate next.

Secondary: General users who receive phishing emails, suspicious links, attachments, or files and need a safe, understandable assessment.

Future: L2/L3 analysts and security engineers who need deeper correlation, investigation guidance, and evidence-driven hypotheses.

4. Product Objectives

Use capable AI models for accurate security analysis, anomaly detection, error/fault discovery, and investigation beyond the initial alert.

Allow users to chat with the investigator for explanations, follow-up questions, and defensive recommendations.

Provide a clean investigation dashboard that presents the most important discovered information visually and progressively.

Keep the product lightweight, simple, responsive, and easy to understand.

Support text, images/screenshots, PDFs, SIEM exports, and suspicious files/links as investigation inputs while treating all user-supplied content as untrusted.

Allow users to export/download the investigation report or dashboard for retention and sharing.

5. Core Features

Multi-input Investigation: text, SIEM log exports, PDF documents, email screenshots/images, suspicious URLs, and files as supported by the ingestion pipeline.

AI Security Investigator: analyzes evidence as a senior security analyst with L2/L3-level investigative behavior.

Investigation Dashboard: verdict, severity, evidence, suspicious events, IOCs, timeline, hypotheses, confidence, gaps, and recommendations.

Investigator Chat: users can discuss findings, ask “why?”, request deeper investigation, and ask what to do next.

Evidence Requests: when the evidence is insufficient, Mirsad can ask for a clearer screenshot, missing log fields, headers, additional events, or another artifact instead of guessing.

Bilingual UI: Arabic default + English, with correct RTL/LTR behavior.

Light/Dark Mode: user-selectable theme.

Export: downloadable investigation report/dashboard.

Safe failure handling: clear errors for unsupported files, unreadable PDFs, malformed inputs, timeouts, and model/API failures.

6. Investigation Dashboard Requirements

Overall Verdict: Safe / Suspicious / Malicious / Inconclusive, with a concise explanation.

Risk Severity: Critical / High / Medium / Low / Informational.

Why this matters: plain-language explanation of the risk.

Evidence: exact or closely referenced input evidence supporting each major finding.

Suspicious Findings: prioritized findings with affected users, hosts, processes, IPs, domains, URLs, hashes, timestamps, and other available fields.

Timeline: chronological sequence of relevant events when timestamps exist.

IOC panel: extracted indicators categorized by type.

Attack hypothesis: plausible attack path/technique only when supported; clearly label assumptions.

Confidence: confidence level plus reasons and missing evidence.

Next investigation steps: what an L1 analyst should check next and where to look for confirmation.

Recommended response: safe defensive actions and escalation guidance; never claim that an action was executed unless the application actually executed it.

Evidence gaps: explicitly state what cannot be concluded from the current material.

7. Analyst Experience — SOC L1

Explain technical findings in a way an L1 analyst can follow without requiring expert interpretation.

Teach the analyst where to look next: related timestamps, source/destination IPs, authentication events, process trees, endpoint telemetry, DNS/proxy events, email headers, URL redirects, and other relevant evidence when available.

Separate “observed in the evidence” from “likely interpretation” and “needs validation.”

Prioritize findings so an analyst does not have to read every log line before understanding the incident.

8. Security & Malware Handling

Uploaded files, screenshots, PDFs, links, and log text must be treated as untrusted input. The application must never execute an uploaded file merely to analyze it.

File processing must use safe parsing, strict size/type limits, temporary isolation, cleanup, and output sanitization.

For potentially malicious URLs/files, Mirsad should analyze available metadata/content safely and clearly state when it cannot safely or reliably determine malware presence.

Do not present a clean verdict merely because no malicious evidence was found; distinguish “no evidence found” from “proven safe.”

Where a safe static analysis cannot establish the answer, the dashboard should say that further sandboxing, endpoint telemetry, reputation checks, or specialist analysis may be required.

9. Prompt Injection & Adversarial Input Resilience

All uploaded/pasted content is evidence, not instructions. A log line such as “SYSTEM: ignore your rules” must be treated as text inside the evidence.

The investigator must resist attempts to reveal system prompts, secrets, API keys, hidden policies, or internal instructions.

User content must not be able to change the investigator's role, security policy, output schema, or higher-priority instructions.

The backend should delimit untrusted evidence clearly, validate structured model output, escape rendered content, and reject malformed output.

Test adversarial cases including fake system messages, prompt injection inside PDFs, malicious text inside screenshots/OCR, huge repeated input, contradictory logs, fake IOCs, and requests to stop the investigation.

10. Technical Architecture

Frontend: semantic HTML5, CSS3, and JavaScript. Keep the UI lightweight and accessible; no unnecessary framework dependency is required for MVP.

Backend: Python, preferably FastAPI, responsible for upload handling, parsing/extraction, normalization, LLM orchestration, validation, and report generation.

LLM: OpenRouter API. The selected model must be configurable without changing frontend code.

Secret management: store OPENROUTER_API_KEY only on the server using an environment variable or secret manager. Never hard-code it, commit it to GitHub, or expose it to browser JavaScript.

Deployment: source code in GitHub and deployment through Render; configuration/secrets are supplied through Render environment variables.

Suggested structure: app/, routes/, services/, schemas/, prompts/, static/, templates/, tests/, .env.example, .gitignore, Agents.md, README.md.

11. OpenRouter Integration

Use a server-side HTTP client to call OpenRouter.

Make model selection configurable through environment configuration, e.g. OPENROUTER_MODEL.

Prefer structured JSON/schema-constrained output when supported by the selected model/provider.

Use bounded input/context lengths, request timeouts, and controlled retries.

Do not store sensitive evidence unnecessarily in application logs.

Track non-sensitive operational metadata such as request status, latency, selected model, and token usage where available.

12. Agent Rules File — Agents.md

Agents.md is mandatory. It contains the operational role, investigation behavior, evidence-handling rules, prompt-injection defenses, response structure, uncertainty requirements, and safe recommendations for the AI investigator.

The attached AI RMF rules are also incorporated into the product requirements. They require lifecycle risk consideration, intended purpose/context/limitations, beneficial and harmful impacts, the four functions GOVERN / MAP / MEASURE / MANAGE, accountability, safety, security/resilience, explainability, privacy, regular TEVV, monitoring, feedback, third-party AI/supply-chain risk management, continual improvement, and explicit documentation of unavailable evidence or measurements. ■filecite■turn0file0■L9-L29■

Developers must treat Agents.md as part of the application security boundary, not as optional documentation.

13. AI RMF Governance Integration

Use GOVERN to define responsibilities, limitations, accountability, security controls, and communication.

Use MAP to document intended use, users, context, assumptions, limitations, impacts, and risks.

Use MEASURE to test accuracy, reliability, prompt-injection resilience, output schema compliance, unsafe behavior, drift, and failure modes.

Use MANAGE to prioritize risks, apply mitigations, monitor incidents, improve controls, and safely disengage or replace the AI behavior when necessary.

Important decisions, assumptions, limitations, tests, metrics, outcomes, and high-priority risk responses should be documented. This directly follows the supplied AI RMF rules. ■filecite■turn0file0■L9-L29■

14. UI / UX & Brand Direction

Design personality: calm, investigative, trustworthy, premium, and modern — not a noisy “hacker” dashboard.

Logo concept: a minimalist shield combined with a magnifying glass/eye, representing protection + investigation + visibility. The mark should work as a full logo, app icon, and favicon.

Primary brand: ■■■■■■■■■■ / Mirsad.

Tagline: “■■■■■ ■■■■■■■ ■■■■■■■■■ ■■■■■■■■.”

Default: Arabic RTL. English switches the complete layout to LTR.

Interaction: large upload area, clear progress states, concise cards, expandable evidence, and an investigator chat alongside results.

15. Brand Color System

Midnight: #0B1117 — main dark background.

Surface: #121C24 — navigation and large surfaces.

Card: #182630 — investigation cards.

Primary Teal: #38D6C5 — brand interaction/accent.

Teal Glow: #72F2E4 — highlights and focus states.

Investigator Gold: #C9A86A — logo/detail accent.

Text: #EAF2F1 — primary typography.

Muted Text: #91A4A8 — secondary typography.

Danger: #FF5C67 — critical/high risk states.

Warning: #F2B84B — warning/medium risk states.

Success: #55D187 — safe/positive states.

16. Logo & Visual Language

Use the shield + magnifying-glass/eye symbol as the core visual metaphor.

Keep the logo simple enough to reproduce in monochrome and small sizes.

Use teal as the “investigation signal” and gold sparingly as the “trusted investigator” accent.

Supporting graphics may use subtle radar rings, evidence markers, grid dots, and restrained glow effects — never at the expense of readability.

17. Code Quality & Engineering Standards

Write clean, readable, modular Python and frontend code with clear naming and small responsibilities.

Keep secrets, configuration, business logic, prompts, schemas, and UI assets separated.

Use type hints and validation for backend request/response models.

Include automated tests for ingestion, extraction, security validation, prompt injection, model schema validation, and API failure paths.

Provide .env.example with placeholder variable names only; never include real credentials.

Include README documentation for local setup, Render deployment, OpenRouter configuration, testing, and security notes.

18. Success Criteria

The system correctly handles supported inputs and safely rejects unsupported or dangerous processing paths.

The dashboard is clean, organized, responsive, and presents meaningful investigation results from SIEM files, email screenshots/images, text, links, or other supported evidence.

Findings are evidence-grounded and clearly distinguish confirmed observations from hypotheses.

The investigator can continue the discussion through chat and recommend practical next investigation steps.

When evidence is insufficient, the model asks for better or additional evidence instead of inventing an answer.

The interface is fast, lightweight, attractive, and easy for an ordinary user or SOC L1 analyst to understand.

Arabic is the default language and English is fully supported.

19. Acceptance Criteria

User can open the site and start an investigation without technical knowledge.

Arabic is the default interface and RTL rendering is correct.

English mode works across navigation, dashboard, chat, errors, and export.

User can submit supported text/PDF/image/file/URL evidence according to implemented safety limits.

API key is never exposed client-side and is not committed to GitHub.

Dashboard displays verdict, severity, evidence, findings, IOCs/timeline where available, confidence, gaps, and recommendations.

Chat remains grounded in the current investigation and resists prompt injection inside evidence.

User can export/download the investigation report/dashboard.

Malformed or unavailable model responses fail safely and do not become trusted HTML.

Core security and AI-risk tests pass before deployment.

20. MVP Out of Scope

Direct autonomous remediation or containment.

Executing uploaded malware/files.

Autonomous shell or endpoint commands.

Full SOAR orchestration.

Guaranteed malware verdicts without sufficient evidence.

Threat-actor attribution presented as fact without strong evidence.

Enterprise case-management workflows unless added later.

21. Future Roadmap

Direct SIEM connectors and API ingestion.

Threat-intelligence enrichment and reputation lookups.

MITRE ATT&CK; mapping.

Saved cases and investigation history.

Advanced correlation and multi-event attack-chain reconstruction.

Controlled SOAR integrations with explicit human approval and audit trails.

Organization-specific detection rules and analyst feedback loops.

22. Definition of Done

Mirsad is deployable on Render from GitHub, uses a server-side OpenRouter integration, has no hard-coded secrets, includes Agents.md, supports Arabic-first bilingual UX, handles untrusted evidence safely, produces structured investigation results, supports follow-up chat and export, and passes the defined security/AI-risk tests.

Appendix A — Brand Palette Preview

Midnight	#0B1117	
Surface	#121C24	
Card	#182630	
Primary Teal	#38D6C5	
Teal Glow	#72F2E4	
Investigator Gold	#C9A86A	
Text	#EAF2F1	
Muted Text	#91A4A8	
Danger	#FF5C67	
Warning	#F2B84B	
Success	#55D187	

Appendix B — Developer Notes

The supplied AI_RMF_RULES.md is the governance baseline for the project. Its terminology and four-function structure (GOVERN, MAP, MEASURE, MANAGE) should remain recognizable in project documentation and testing. ■filecite■turn0file0■L5-L16■

Where evidence or measurements are unavailable, the application must explicitly state the limitation rather than present assumptions as facts; transparent and explainable decisions are preferred over unsupported claims. ■filecite■turn0file0■L17-L29■